Pair Programming

Pair programming is an Agile software development technique originating from **Extreme programming (XP)** in which two developers team together on one computer. The two people work together to design, code and test user stories. Ideally, the two people would be equally skilled and would each have equal time at the keyboard.

In Extreme Programming (XP), the term "Extreme" refers to taking best software development practices [mentioned below] to an extreme level to enhance efficiency, quality, and flexibility.

Code Reviews → XP takes it to the extreme with pair programming (continuous review).

Testing → XP emphasizes Test-Driven Development (TDD), writing tests before writing code.

Simplicity → XP encourages simplest solutions first, avoiding unnecessary complexity.

Feedback → XP relies on rapid and continuous feedback from customers and team members.

The idea is that by pushing these good practices to their limits, software teams can produce higher-quality software with greater adaptability

A common implementation of pair programming calls the programmer at the keyboard the *driver*, while the other is called the *navigator*. The navigator focuses on the overall direction of the programming. The collaboration between developers can be done in person or remotely.

Pair programming is a collaborative effort that involves a lot of communication. The idea is to have the driver and navigator communicate, discuss approaches and solve issues that might be difficult for a single developer to detect.

Agile software development technique is not well suited for everyone, however, learning to partner effectively in a team that close and share a work computer takes skills that not all programmers possess. It requires both programmers to have the soft skills required for collaboration, as well as the requisite hard skills to write and test code.

How does pair programming work?

Pair programming requires two developers, one workstation, one keyboard and a mouse. The pairings can be assigned or self-assigned.

Pair programming uses the four eyes principle, which ensures two sets of eyes review the code that is being produced. While the driver writes the code, the navigator checks the code being written. The driver focuses on the specifics of coding, while the navigator checks the work, code quality and provides direction.

The two developers take turns coding or reviewing and check each other's work as they go. Rotating roles regularly helps keep both developers alert and engaged. Organizations may also have the **pair rotate** roles to work on different tasks. This way, they get experience working on the different parts of the system being built.

Depending on how the pairs are coordinated, junior and senior developers can work together, enabling senior developers to share their knowledge and working habits. It also helps new team members get up to speed on a project.

Benefits of pair programming

Pair programming includes the following advantages:

- **Fewer coding mistakes**. Another programmer is looking over the driver's code, which can help reduce mistakes and improve the quality of the code.
- **Knowledge is spread among the pairs**. Junior developers can pick up more skills from senior developers. And those unfamiliar with a process can be paired with someone who knows more about the process.
- **Reduced effort to coordinate.** Developers will get used to collaborating and coordinating their efforts.

- **Increased resiliency**. Pair programming helps developers understand each part codebase, meaning the environment will not be dependent on a single person for repairs if something breaks.

Challenges of pair programming

Pair programming includes the following challenges

- **Efficiency.** Common logic might dictate that pair programming would reduce productivity by 50%, because two developers are working on the same project at one time. According to a blog post on Raygun, pairs work about 15% slower, which is an improvement but is still less than the productivity of two separate programmers.
- **Equally engaging pairs**. If both developers do not equally engage in the project, then there is less chance that knowledge will be shared, and it is highly probable that one developer will participate less than the other.

- **Social and interactive process**. It is hard for those who work better alone. Pairs that have trouble together might be better suited to work by themselves; forcing them to collaborate may hurt their work ethic.
- **Sustainability**. The pace may not be suited to practicing hours at a time. Likely, developers will need breaks at different times.

Pair programming styles and techniques

Pair programming uses three general programming styles: the driver/navigator style, unstructured style and ping-pong style.

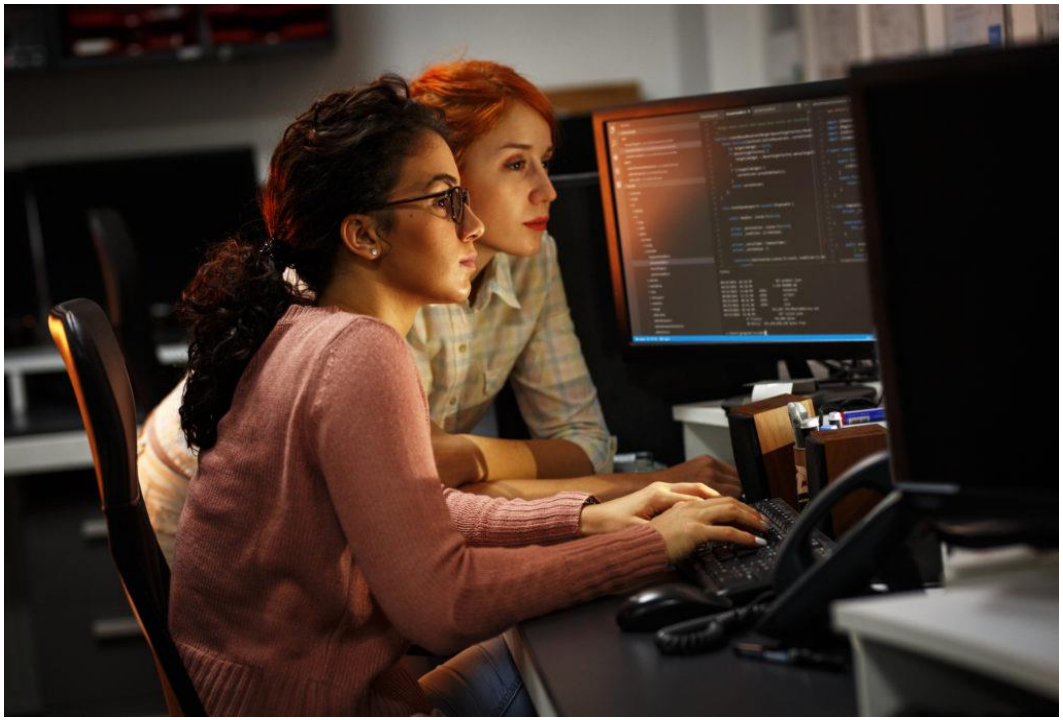
Driver/navigator style. The driver/navigator approach is a popular pair programming style where one programmer handles the mechanical side like coding, and the other is in control of the strategic or architectural elements like reviewing code. The driver and navigator switch roles often. This style works well for a novice paired with an expert programmer.

Unstructured style. Most pair programming relationships fall into the unstructured style, where two programmers work together in an Ad hoc manner and collaboration is loosely guided. Both programmers should have matching skill levels. A common variant of this style is the

unstructured expert-novice pair, where an expert programmer and a novice are paired together.

An unstructured approach is difficult to discipline and unlikely to last for longer projects. Unstructured pair programming is also harder to keep afloat remotely.

Ping-pong style. With the ping-pong approach, one developer writes a test and the other developer makes the test pass. Each person alternates between writing and passing tests. When two developers shift roles regularly, it is unlikely one programmer will control the workflow. This style of pair programming is normally performed in conjunction with test-driven development.



Development teams should also choose pair programming styles that align with the skills of the programmers involved. Potential skill pairs include:

Expert/expert pairs. Two experts can generally work within any pair programming style. Advanced programmers may prefer the ping-pong style, as it allows them to have even participation.

Novice/novice pairs. Two novices together may have difficulty in the driver/navigator style, because no one is experienced enough to take charge. In addition, the unstructured approach may be difficult for beginner programmers.

Expert/novice pairs. The most common skill combination is an expert programmer working with a less experienced person. Experts rely on their depth of knowledge to direct the activity, while the novice can learn more from the expert.

Best practices for pair programming

Some best practices to follow when enacting pair programming include:

- **Consistent communication**. If there is no communication going on, then the developers are likely not sharing thought processes.
- **Switch roles consistently**. This promotes the sharing of more skills between developers and keeps them engaged.
- **Pair up carefully**. Ensure the two developers will be able to work together well, without any hiccups. Otherwise, this will make for a poor work environment.
- **Use a familiar development environment**. Both developers should be familiar with the environment they are working in. Otherwise, the balance of pair programming will be disrupted.

- **Submit code frequently**. Quick commits to code pair well with switching between the driver and the navigator.

- **Ask for clarification when needed**. Particularly when a novice works with an expert, the novice should take any opportunity to learn.

- **Take breaks when needed.** Work at a pace that fits both developers.

Emergence of remote pair programming

Remote pair programming developers work together using a variety of tools, such as a collaborative real-time editor, shared desktops or a remote pair programming integrated development environment (IDE) plugin. Remote pairing can introduce complexities such as extra delays in coordination, a potential loss in communication and an increased reliance on task-tracking tools.

Collaborative coding tools include two pieces of technology: a communications channel and an IDE. The collaborative communications tool can be a real-time audio or video tool, such as a Microsoft Teams video call or Zoom. The collaborative IDE tool enables the programming pair to share access to code and make references, such as with Microsoft Visual Studio and CodeSandbox with its Live feature.

Developer pairs should schedule meetings each week for the same day and time in order to establish the objectives of each pair programming session before it starts. If a team is just moving to remote pair programming, then extra time should be allocated to work out any kinks and try different styles.